

Module : Kotlin (M202) TP N° 18

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVOAM101 - DEVOAM102 Niveau : 2ème année

1. Projet sans Hilt (Gestion manuelle des dépendances)

Structure du projet :

- **MainActivity.kt** (Activity principale)
- **Repository.kt** (Un dépôt de données simulé)
- **Service.kt** (Un service pour fournir des données)

Service.kt (service simulé qui renvoie des données)

```
class Service {  
  
    fun getData(): String {  
  
        return "Données fournies par le service"  
  
    }  
  
}
```

Repository.kt (dépôt qui utilise le service)

```
class Repository(private val service: Service) {  
  
    fun fetchData(): String {  
  
        return service.getData()  
  
    }  
  
}
```

MainActivity.kt (Activity principale)

```
import androidx.appcompat.app.AppCompatActivity

import android.os.Bundle

import android.widget.TextView

class MainActivity : AppCompatActivity() {

    // On crée manuellement les dépendances

    private val service = Service()

    private val repository = Repository(service)

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_main)

        val textView: TextView = findViewById(R.id.textView)

        // On récupère les données depuis le repository

        textView.text = repository.fetchData()

    }
```

}

- Ici, on crée manuellement une instance de **Service** et on la passe à **Repository**.
- Ensuite, dans **MainActivity**, on crée une instance de **Repository** et on appelle ses méthodes.
- Cette méthode fonctionne bien pour des projets simples, mais elle peut devenir difficile à gérer à mesure que le projet devient plus complexe. Chaque dépendance doit être gérée manuellement.

2. Projet avec Hilt

Hilt simplifie la gestion des dépendances en fournissant automatiquement les instances nécessaires à travers des annotations. Voici comment on peut réécrire l'exemple précédent en utilisant Hilt.

Ajouter Hilt à ton projet

Dans le fichier **build.gradle** du projet, ajoute les dépendances nécessaires à Hilt :

```
// build.gradle (module)
```

```
plugins {
```

```
    id 'com.android.application'
```

```
    id 'org.jetbrains.kotlin.android'
```

```
    id 'dagger.hilt.android.plugin'
```

```
}
```

```
android {
```

```
compileSdkVersion 33
```

```
defaultConfig {
```

```
    applicationId "com.example.hiltexample"
```

```
    minSdkVersion 21
```

```
    targetSdkVersion 33
```

```
}
```

```
}
```

```
dependencies {
```

```
    implementation 'com.google.dagger:hilt-android:2.48'
```

```
    kapt 'com.google.dagger:hilt-compiler:2.48'
```

```
}
```

```
// build.gradle (projet)
```

```
buildscript {
```

```
    dependencies {
```

```
        classpath "com.google.dagger:hilt-android-gradle-plugin:2.48"
```

```
    }
```

```
}
```

MainActivity.kt (avec Hilt)

```
import androidx.appcompat.app.AppCompatActivity

import android.os.Bundle

import android.widget.TextView

import dagger.hilt.android.AndroidEntryPoint

import javax.inject.Inject

@AndroidEntryPoint

class MainActivity : AppCompatActivity() {

    // Hilt injecte automatiquement le Repository

    @Inject

    lateinit var repository: Repository

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_main)

        val textView: TextView = findViewById(R.id.textView)

        // Utilisation du Repository injecté pour afficher les données

        textView.text = repository.fetchData()

    }

}
```

Repository.kt (avec Hilt)

```
import javax.inject.Inject

class Repository @Inject constructor(private val service: Service) {

    fun fetchData(): String {

        return service.getData()

    }

}
```

Service.kt (avec Hilt)

```
import javax.inject.Inject

class Service @Inject constructor() {

    fun getData(): String {

        return "Données fournies par le service via Hilt"

    }

}
```

App.kt (Application avec Hilt)

N'oublie pas de configurer ta classe Application pour Hilt :

```
import android.app.Application

import dagger.hilt.android.HiltAndroidApp

@HiltAndroidApp
```

```
class App : Application()
```

- Grâce à Hilt, nous n'avons plus besoin de créer manuellement les instances des classes dans `MainActivity`. Hilt gère cela automatiquement.
- `@Inject` sur les constructeurs de `Service` et `Repository` indique à Hilt comment créer ces objets.
- `@AndroidEntryPoint` dans `MainActivity` permet à Hilt d'injecter les dépendances dans l'Activity.
- La classe `App` est annotée avec `@HiltAndroidApp` pour initialiser Hilt dans l'application.